ASU W. P. Carey School of Business
Arizona State University

HCLSoftware

Associated Remediation for Scan Findings

Team 461

Garvita Jain | Justin Tai | Min Kim | Rakshit Vasava | Prathamesh Awati

Agenda

1. Project Objective
2. Data & Methodology
3. Model Selection
    a. Model 1: **QA - Haystack**
    b. Model 2: **GPT2 - HuggingFace**
    c. Model 3: **ChatOpenAI - Langchain**
4. Model Comparison
5. Key Takeaway
6. Recommendation
7. Future Scope
8. References
9. Questions

PROJECT OBJECTIVE

To use **Large Language Models (LLMs)** to provide precise, context-specific code solutions for software vulnerabilities that HCL AppScan software had found.

By combining user feedback and guaranteeing high accuracy with little confusion, the project aims to innovate code remediation by moving from conventional code examples to sophisticated, automated code creation solutions.

Data & Methodology

SOURCE OF DATA

1. **HCL Software's database**
2. **OWASP Cheat Sheet API**

for up-to-date security practices and remediation strategies

TYPE OF DATA

Descriptions of security issues

Remediation advice

Examples of vulnerable code

METHODOLOGY

Input: **Vulnerability, Context**

Large Language Model

Output: **Articles, Recommendation**

Model Selection

haystack by deepset

Specializes in extracting precise answers from complex datasets, enhancing **question-answering tasks** within the Haystack framework for efficient and accurate information retrieval.

Hugging Face

Utilizes transformer architecture to effectively process and generate **context-aware text**, improving relevance and accuracy in text-based applications like code fixes and content generation.

LangChain

Integrates **retrieval and generation** to deliver detailed, accurate, and context-aware responses, making it ideal for complex querying in applications such as customer support and content creation

Modeling 1: *QA - Haystack*

Q: What is Cross Site Scripting?

```
'Answers:'
[   {   'answer': 'Prevention',
        'context': ' the document. The very first OWASP Cheat Sheet, Cross '
                   'Site\n'
                   "Scripting Prevention, was inspired by RSnake's work and we "
                   'thank RSnake for\n'
                   'the inspiratio',
        'score': 0.7371946573257446},
    {   'answer': 'a type of attack where malicious JavaScript code\n'
                  'is injected into a displayed variable',
        'context': '\n'
                   '\n'
                   'Cross-Site Scripting (XSS) is a type of attack where '
                   'malicious JavaScript code\n'
                   'is injected into a displayed variable. For example, if the '
                   'value of t',
        'score': 0.5444809198379517},
    {   'answer': 'testing for application\nsecurity professionals',
        'context': 'is article is a guide to Cross Site Scripting (XSS) '
                   'testing for application\n'
                   'security professionals. This cheat sheet was originally '
                   "based on RSnake's\n",
        'score': 0.4696384072303772},
    {   'answer': 'Prevention Cheat Sheet',
        'context': 'ext of an XSS\n'
                   'attack.\n'
                   '\n'
                   'See the OWASP XSS (Cross Site Scripting) Prevention Cheat '
                   'Sheet.\n'
                   '\n'
                   'See also: HttpOnly\n'
                   '\n'
                   '### SameSite Attribute\n'
                   '\n'
                   'SameSite defines ',
        'score': 0.3799809217453003},
    {   'answer': 'XSS) attacks',
        'context': ' pages from loading when they detect reflected cross-site '
                   'scripting (XSS) attacks. This header has been deprecated '
                   'by modern browsers and its use can ',
        'score': 0.36732858419418335}]
```

- **Baseline Model:** **Question Answering System** using Haystack's DocumentStore, Retriever, and Reader
- **Model 1**: Trained on the *dataset provided by HCL Team*, this version excels in processing client-specific data, adept at delivering precise responses relevant to the provided information.
- **Model 2**: Incorporating the *OWASP dataset* improves the system's ability to address cybersecurity queries, especially on web vulnerabilities, showcasing advanced understanding and solution generation.

Modeling 2: *GPT2 - HuggingFace*

Q: What is Cross Site Scripting (XSS)?

```
  * **Cross site scripting injection (XSS) refers to a virus, malware,
script, JavaScript, etc. that attacks a resource on an
application, then instructing that the resources are loaded from a
different location, causing the
application to inject itself into the victim server;**
  * **XSS refers to a malicious code in a data source system, such as an
application, server, phone, or computer, that attacks a
resource on the victim server.**

In general, this is a relatively minor threat, but if you are
understandable as a developer and know enough to know where the problem
is, consider implementing
measures to prevent cross site scripting.

## 2.2 - Avoid using JavaScript by Appropriate Paths

It may well not be a good idea to enable JavaScript paths in the `!eval`
document input when executing scripts, or to use a different origin from
the
`!evalPath` directive. Such directives need to be enabled and enforced to
prevent
cross site scripting attacks like Cross Site Scripting (XSS).

The simplest way to do so is to disable JavaScript paths after defining
the
parameter `!evalPath` in the `...'&$'` directive:

  * **Always return `!evalPath` or `!evalN` tags in `!alert(xhr=4)'` or
```

- **Baseline Model:** GPT2
- **Model 1**: Trained on a *dataset provided by HCL Team*, this model frequently generated code snippets, mirroring the dataset's nature, which primarily includes JavaScript snippets.
- **Model 2:** Training on the *OWASP dataset* led to higher quality outputs, with fewer code snippets, indicating a better grasp of coherent sentence formation.

Modeling 3: *ChatOpenAI - Langchain*

Q: What is Cross Site Scripting? And how to solve it?

```
Response: Cross-Site Scripting (XSS) is a type of attack where malicious Java
Script code is injected into a displayed variable. To prevent XSS vulnerabili
ties, developers should sanitize user input by filtering out any potentially
harmful characters such as <, >, &, ", ', and /.

One way to solve XSS vulnerabilities is to use input validation and output en
coding. Input validation ensures that user input meets certain criteria, such
 as only allowing alphanumeric characters. Output encoding involves encoding
user input before displaying it on a webpage, so that any potentially harmful
 characters are rendered harmless.

Another solution is to implement Content Security Policy (CSP) headers on web
 servers. CSP allows developers to specify which sources of content are allow
ed to be loaded on a webpage, thereby preventing the execution of malicious s
cripts.

By implementing these solutions, developers can effectively prevent XSS attac
ks and protect their applications from malicious code injections.
Sources: ['data/cheatsheets/Symfony_Cheat_Sheet.md', 'data/cheatsheets/Symfon
y_Cheat_Sheet.md', 'data/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sh
eet.md']
```

- **Baseline Mode:** ChatOpenAI
- **Model:** Training on the *OWASP dataset* and create a new database using Chroma. The dataset is stored in form of vectors, which makes it easier for the model to process the data.

Modeling 3: *ChatOpenAI - Langchain*

Q: What is SQL injection? And how to solve it?

```
Response: SQL injection is a type of security vulnerability that occurs when
an attacker is able to manipulate a SQL query in a way that it can execute ar
bitrary SQL code. This can allow attackers to view, modify, or delete data in
 the database, potentially leading to unauthorized access or data loss.

To solve SQL injection vulnerabilities, developers should use parameterized q
ueries or prepared statements. This involves separating the SQL query from th
e user input and using placeholders for the input values. By binding the inpu
t values to the placeholders, the SQL query is executed with the input values
 treated as data rather than executable code.

It is also important to validate and sanitize user input to ensure that it do
es not contain any malicious SQL code. Input validation can help prevent atta
ckers from injecting SQL code into the application.

By following these practices, developers can protect their applications from
SQL injection attacks and ensure the security of their databases.
Sources: ['data/cheatsheets/Symfony_Cheat_Sheet.md', 'data/cheatsheets/Inject
ion_Prevention_Cheat_Sheet.md', 'data/cheatsheets/Injection_Prevention_Cheat_
Sheet.md']
```

- **Baseline Mode:** ChatOpenAI
- **Model:** Training on the *OWASP dataset* and create a new database using Chroma. The dataset is stored in form of vectors, which makes it easier for the model to process the data.

Model Comparison

ChatOpenAI > GPT2 > QA

ChatOpenAI model

```
Response: Cross-Site Scripting (XSS) is a type of attack where malicious Java
Script code is injected into a displayed variable. To prevent XSS vulnerabili
ties, developers should sanitize user input by filtering out any potentially
harmful characters such as <, >, &, ", ', and /.

One way to solve XSS vulnerabilities is to use input validation and output en
coding. Input validation ensures that user input meets certain criteria, such
 as only allowing alphanumeric characters. Output encoding involves encoding
user input before displaying it on a webpage, so that any potentially harmful
 characters are rendered harmless.

Another solution is to implement Content Security Policy (CSP) headers on web
 servers. CSP allows developers to specify which sources of content are allow
ed to be loaded on a webpage, thereby preventing the execution of malicious s
cripts.

By implementing these solutions, developers can effectively prevent XSS attac
ks and protect their applications from malicious code injections.
Sources: ['data/cheatsheets/Symfony_Cheat_Sheet.md', 'data/cheatsheets/Symfon
y_Cheat_Sheet.md', 'data/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sh
eet.md']
```

OWASP Cheat Sheet

Reliance on HTTP Interceptors

The other common anti-pattern that we have observed is the attempt to deal with validation and/or output encoding in some sort of interceptor such as a Spring Interceptor that generally implements `org.springframework.web.servlet.HandlerInterceptor` or as a JavaEE servlet filter that implements `javax.servlet.Filter`. While this can be successful for very specific applications (for instance, if you validate that all the input requests that are ever rendered are only alphanumeric data), it violates the major tenet of XSS defense where perform output encoding as close to where the data is rendered is possible. Generally, the HTTP request is

Other Controls

Framework Security Protections, Output Encoding, and HTML Sanitization will provide the best protection for your application. OWASP recommends these in all circumstances.

Consider adopting the following controls in addition to the above.

- Cookie Attributes - These change how JavaScript and browsers can interact with cookies. Cookie attributes try to limit the impact of an XSS attack but don't prevent the execution of malicious content or address the root cause of the vulnerability.
- Content Security Policy An allowlist that prevents content being loaded. It's easy to make mistakes with the implementation so it should not be your primary defense mechanism. Use a CSP as an additional layer of defense and have a look at the cheatsheet here.

KEY TAKEAWAYS

- Fine-tuning a pre-trained Large Language Model demands **structured input training datasets**, such as Question-Answer or Input-Output pairs.
- To facilitate the automation of code vulnerability detection and correction, we utilized datasets from HCL Software and the OWASP API, employing ***QA***, ***GPT2***, and ***ChatOpenAI*** model.
- However, we faced significant **computational and economic challenges** due to our inability to format the training datasets appropriately for each model.

Recommendation

- To overcome these obstacles, we propose three innovative, parameter-efficient fine-tuning techniques designed to streamline our project more effectively:

1

Parameter-Efficient Fine-Tuning with Adaptors

1. **What**: Adds small, trainable adapters to pre-trained models
2. **Why**: Saves resources by avoiding full retraining
3. **How**: Fine-tune **adapters** only, keeping the rest fixed

2

Domain and Task Adaptive Fine-Tuning

1. **What**: Customizes models with targeted data augmentation
2. **Why**: Boosts performance by enhancing task/domain relevance
3. **How**: Use specific **data augmentation** to improve specialization

3

Direct Preference Optimization (DPO)

1. **What**: Aligns model tuning with human preferences
2. **Why**: Streamlines training and improves output relevance
3. **How**: Directly optimize using a **dataset of human preferences**

Future Scope

- Our model can be extended in following ways:

1

Chatbot Integration in Development Tools:

Embed intelligent chatbots for instant platform-solving and enhanced developer efficiency.

2

Model Expansion for Diverse Queries:

Evolve the model to address a broader range of system queries, increasing its utility and adaptability.

3

Partnership with OpenAI for a Comprehensive Platform:

Join forces with OpenAI to develop a robust software ecosystem that supports users at every platform stage.

References

- QA Documentation: https://haystack.deepset.ai/tutorials/01_basic_qa_pipeline
- GPT2 Documentation: https://huggingface.co/openai-community/gpt2
- Langchain RAG Tutorial: https://github.com/pixegami/langchain-rag-tutorial
- OWASP Cheat Sheet Series: https://cheatsheetseries.owasp.org/index.html

Any Questions?

Thank you